C++ Exception Handling

C++ is a powerful language that provides robust mechanisms to handle unexpected situations in a program. One such mechanism is **exception handling**. This tutorial will guide you through the concepts, syntax, standard exception classes, and best practices in C++ exception handling.

What is Exception Handling?

Exception handling is the process of responding to exceptional (unexpected or error) conditions that occur during the execution of a program. Instead of crashing, a C++ program can catch these exceptions and handle them gracefully.

C++ uses three main keywords for exception handling:

- `try` – Defines a block of code in which exceptions may occur.
- `throw` – Used to throw an exception.
- `catch` – Defines a block of code to handle the exception.

Basic Syntax

```
try
{
    // Code that might throw an exception, using `throw` keyword
}
catch (exception_type variable)
{
    // Code to handle the exception
}
```

Divide by Zero

```
#include <iostream>
using namespace std;

int main()
{
```

```
    int numerator = 10;
    int denominator = 0;

    try
    {
        if (denominator == 0)
            throw "Division by zero error!";
        cout << "Result: " << numerator / denominator << endl;
    }
    catch (const char* msg)
    {
        cout << "Caught Exception: " << msg << endl;
    }

    return 0;
}
```

Output:

```
Caught Exception: Division by zero error!
```

Multiple Catch Blocks

C++ allows multiple `catch` blocks to handle different types of exceptions.

```
try
{
    // some code
    throw 100;
}
catch (int e)
{
    cout << "Caught integer: " << e << endl;
}
catch (const char* msg)
{
```

```
    cout << "Caught message: " << msg << endl;
}
```

Catch-All Handler

You can use a catch-all block to handle any type of exception:

```
try
{
    throw 3.14;
}
catch (...)
{
    cout << "Caught an exception of unknown type" << endl;
}
```

Custom Exception Class

You can create your own exception class:

```
#include <iostream>
using namespace std;

class MyException
{
public:
    const char* what() const
    {
        return "Custom exception occurred!";
    }
};

int main() {
    try
    {
        throw MyException();
```

```
    }
    catch (MyException& e)
    {
        cout << e.what() << endl;
    }
    return 0;
}
```

Standard Exceptions

C++ provides several standard exception classes in the `<stdexcept>` and `<exception>` headers:

- `std::exception` : Base class
- `std::logic_error` , `std::runtime_error` : Derived classes
- `std::invalid_argument` , `std::out_of_range` , `std::overflow_error` , etc.

Example Using `std::runtime_error`

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main()
{
    try
    {
        throw runtime_error("Runtime error occurred!");
    }
    catch (const exception& e)
    {
        cout << "Caught: " << e.what() << endl;
    }
    return 0;
}
```

More Examples

Handling Multiple Types

```
#include <iostream>
using namespace std;

int main()
{
    try
    {
        int a = 5;
        if (a == 5)
            throw 'A';
        else
            throw 100;
    }
    catch (int x)
    {
        cout << "Caught an integer: " << x << endl;
    }
    catch (char y)
    {
        cout << "Caught a character: " << y << endl;
    }
    return 0;
}
```

Nested Try-Catch Blocks

```
#include <iostream>
using namespace std;

int main()
{
    try
    {
        cout << "Outer try block\n";
```

```
        try
        {
            cout << "Inner try block\n";
            throw 10;
        }
        catch (int x)
        {
            cout << "Caught in inner catch: " << x << endl;
            throw; // rethrow to outer catch
        }
    }
    catch (...)
    {
        cout << "Caught in outer catch\n";
    }
    return 0;
}
```

Custom Exception derived from std::exception

```
#include <iostream>
using namespace std;

class MyException
{
public:
    const char* what() const
    {
        return "My custom exception!";
    }
};

int main() {
    try
    {
        throw MyException();
    }
```

```
    catch (const MyException& e)
    {
        cout << "Caught: " << e.what() << endl;
    }

    return 0;
}
```

Array Index out of Bounds

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main()
{
    int arr[5] = {1, 2, 3, 4, 5};
    int index = 7;

    try
    {
        if (index < 0 || index >= 5)
        {
            throw out_of_range("Array index out of bounds");
        }
        cout << "Value at index " << index << ": " << arr[index] << endl;
    }
    catch (const out_of_range& e)
    {
        cout << "Caught exception: " << e.what() << endl;
    }
    return 0;
}
```

Catch-All Handler

```cpp
#include <iostream>
using namespace std;

int main()
{
    try
    {
        throw 42;  // Throwing an integer exception
    }
    catch (int x)
    {
        cout << "Caught integer: " << x << endl;
    }
    catch (...)
    {
        cout << "Caught an exception of unknown type" << endl;
    }

    return 0;
}
```